VR-CSI Project Context

Full architecture, conventions, and extension guide
for AI contributors

Unity 6000.5.6f1 · URP · Meta XR / Oculus Interaction SDK · MCP for Unity

Generated 2026-09-12 from a long-running development session.
Project root: `C:\Users\Roduf Eleu\My project (1)`

Contents

1. Project Overview
2. File Structure
3. Core Architecture
  3.1 EvidenceStateManager — the state machine
  3.2 EvidenceProp / EvidenceDefinition — the data model
  3.3 PlayerTool / the tool wheel
  3.4 The marker precedent pattern
  3.5 On-scene fingerprint sub-step (FingerprintKitTool/FingerprintTapeTool)
  3.6 Scene-authored vs runtime-generated objects (EvidenceChalkGuide)
  3.7 PhotographTool aim-target resolution
  3.8 GreyboxFlowTest
  3.9 GrabbableEvidenceBase.prefab
  3.10 Assessment pipeline (SessionLogger, ScenarioGroundTruthExporter, EvidenceScorer)
  3.11 InteractionFeedback
  3.12 Hypothesis / Deduction system
  3.13 STCS system
  3.14 RecorderReportUI & cross-item unlocks
4. Established Conventions and Gotchas
5. How to Extend This Project
6. Current Known Gaps / Open Items

1. Project Overview

What this is. A VR crime-scene-investigation (CSI) training and *assessment* scenario, built as a thesis project. The player is a single unified investigator (not a multi-role team) who walks through a scene (currently a house, scenario `CSI_Environment`), finds evidence, documents it through a real forensic procedure, and submits a conclusion. The point is not "collect all the evidence" — it is measuring whether the player followed the *correct real-world procedure* and drew *correct conclusions*, so the whole design is built around not leaking hints.

Design philosophy (load-bearing, not decorative)

- **Free placement, no hints.** Marking a numbered evidence tent on a decoy produces the exact same feedback (audio/haptic) as marking the real murder weapon — see `InteractionFeedback` (§3.11) and `EvidenceNotifier`, which deliberately excludes the `Marked` status from its toast list for this reason. Whether an identification was *correct* is only knowable after the fact from the session log, never fed back live.
- **Procedural compliance is the opposite case — it must be loud.** Skipping a required step (trying to collect before swabbing, trying to seal before collecting) is always refused with a clear message and a distinguishable "blocked" haptic/tone. The distinction between these two feedback classes is documented explicitly in `InteractionFeedback`'s class comment and must never be collapsed.
- **The real procedure is the spec.** Almost every ordering decision in this codebase (swab-before-flip, on-scene fingerprint-before-collection, sealing before lab processing) was chosen because that is how a real CSI investigator would have to do it, not for gameplay convenience.
- **Scoring reads the full event log, not a final-state snapshot.** E.g. `TryReclaimMarker` rewinds state but never deletes the original log event — a player who tents everything, watches for a reaction, then quietly un-tents the wrong ones must not be able to erase that signal.

Tech stack

Layer	Detail
Engine	Unity 6000.5.6f1
Render pipeline	URP (Universal Render Pipeline) — generated placeholder materials must always use `Shader.Find("Universal Render Pipeline/Lit")`, never the Built-in "Standard" shader (renders magenta — see §4)
XR	Meta XR / Oculus Interaction SDK (Grabbable, HandGrabInteractable, GrabInteractable), XR Interaction Toolkit input actions (XRI Default Input Actions asset), OVRCameraRig
Editor automation	MCP for Unity (`com.coplaydev.unity-mcp`) — this entire project has been developed via an AI agent driving the Unity Editor through this bridge (`execute_code`, `manage_scene`, etc.), not by a human clicking in the Editor
Backend/logging	Local JSON session logs (`LocalJsonLogWriter`) written to `Application.persistentDataPath`; a standalone .NET 8 console scorer (`Tools/EvidenceScorer`) reads them offline

> **For an AI picking this up cold:** almost every non-trivial design decision in this codebase has a documented "why" in an XML doc comment directly above the code, often referencing a confirmed live bug that motivated it (not a hypothetical). Read those comments before changing behavior — they are the design history, and several of them explicitly warn against a "fix" that was already tried and found wrong.

2. File Structure

Real, current tree under `Assets/_Project` (obtained via Glob, not reconstructed from memory). Grouped by system rather than alphabetically.

Scripts/CaseFile — the evidence data model and state machine

```
EvidenceStateManager.cs the central state machine (see §3.1) EvidenceDefinition.cs ScriptableObject: per-item authored data (see §3.2) EvidenceRecord.cs runtime record per evidenceId (status + per-item flags) EvidenceProp.cs MonoBehaviour marking a scene object as evidence + Found trigger EvidenceStatus.cs the enum (NotFound..Processed) + explicit-ordinal discipline EvidenceRelevance.cs Critical/Relevant/Distractor/Neutral (scoring-only, never gates play) EvidenceSwabRequirement.cs marker precedent: a blood-trace sub-object (see §3.4) EvidenceFingerprintMarker.cs marker precedent: an on-scene print sub-object (see §3.4, 3.5) SelectionPrerequisite.cs cross-item report-line/hypothesis unlock rule (see §3.14)
```

Scripts/Interaction — every tool and scene-interactive object

```
PhotographTool.cs camera; aim-target resolution (see §3.7) ChalkMarkTool.cs continuous trace-based chalk outline tool EvidenceChalkGuide.cs the per-item chalk guide (scene-authored conversion, see §3.6) ChalkGuideSegmentLink.cs live line-follow helper for the chalk guide (see §3.6) EvidenceTentTool.cs numbered evidence tent dispenser LetterTentTool.cs separate tool for the LETTER tent (examinedInPlace items) EvidenceTentPickup.cs / EvidenceTentPickupCoordinator.cs tent reclaim (currently disabled, see §6) SwabTool.cs / SwabTipReceiver.cs combined swab+container, collision-based FingerprintKitTool.cs dust brush (+jar), collision-based, generalized (see §3.5) FingerprintTapeTool.cs / TapeTipReceiver.cs lift tape, separately equippable (see §3.5) FingerprintStationTool.cs DEFAULT (post-Sealed) dust/lift/compare station — EVD-017/018 only now BrushTipReceiver.cs thin OnTrigger forwarder for the dust brush EvidenceFlipTool.cs aim-and-activate flip action (gloves only) EvidenceBagTool.cs / EvidenceBagReceiver.cs physical bag-and-seal collection EvidenceGrabGate.cs disables hand-grab until ReadyForCollection + sub-steps done UVRevealable.cs UV-beam hidden/revealed visibility state machine FlashlightTool.cs UV beam source, raycast-based CurrentBeamTarget RecorderTool.cs aim-and-activate; opens the drag-and-drop report panel RightHandOnlyFilter.cs / LeftHandOnlyFilter.cs Oculus HandGrab interactor filters ToggleGrab.cs physical-grab toggle behavior (on GrabbableEvidenceBase) OpenableFurniture.cs / AtticLadderLever.cs / FoldingStairToggle.cs environment interactables CameraShutterEffect.cs / ViewfinderFrameMask.cs / ViewfinderConfirmationFeedback.cs / AimIndicatorUI.cs / PhotoCaptureListener.cs / HudFollowCamera.cs camera/viewfinder presentation
```

Scripts/RoleSystem — the tool wheel / virtual-equip framework

```
PlayerTool.cs abstract base every tool inherits (see §3.3) PlayerToolRegistry.cs static directory + equip/holster orchestration ToolType.cs explicit-int enum, one entry per equippable capability ToolWheelController.cs the radial wheel UI, auto-enumerates ToolType ToolWheelWedgeTemplate.cs editor-authored wedge template marker LocomotionSuspender.cs pauses movement while the wheel is open RoleSceneLoader.cs / PlayerHeightOffset.cs scene bootstrap / VR comfort
```

Scripts/Deduction — hypothesis checkpoints and the evidence board

```
HypothesisCheckpointManager.cs watches evidence progress, fires authored checkpoints (see §3.12) HypothesisCheckpointSet.cs / HypothesisCheckpointUI.cs data asset type + presentation (has a runtime-generated FALLBACK panel, only built if no panelRoot authored — a pending scene-authoring conversion, see §6) CompletionScreenUI.cs end-of-scenario screen (also has a runtime-generated fallback, see §6) EvidenceBoardController.cs / EvidenceNode.cs / ConnectionLine.cs / DeductionScorer.cs a corkboard-style evidence-connection board (separate from the hypothesis-checkpoint flow; read DeductionScorer before changing either, they are easy to conflate)
```

Scripts/STCS — Scripted Team Communication System

```
STCSManager.cs fires authored dialogue triggers off EvidenceStateManager events (see §3.13) STCSTrigger.cs one trigger entry (id, dialogue pool, side effects) STCSNotificationUI.cs / DialogueLinePool.cs presentation + line-pool data asset
```

Scripts/UI, Feedback, Logging, ProceduralGate, Testing

```
UI/ NotificationManager.cs + NotificationUI.cs (async toast QUEUE, see §4) EvidenceNotifier.cs (bridges OnEvidenceStatusChanged -> generic toasts) RecorderReportUI.cs (drag-and-drop report panel, see §3.14) PlayerUIGate.cs (static blocker-count gate so only one panel owns input at a time) UtilityMenuController.cs, PhotoAlbumManager.cs/UI.cs, ReportLineChip.cs, UIManager.cs Feedback/ InteractionFeedback.cs (Confirm/Blocked channels, see §3.11) FeedbackDirector.cs (higher-level feedback orchestration - read before adding a new "success" cue; it documents which statuses deliberately get NO cue) Logging/ SessionLogger.cs, SessionEvent.cs, SessionEventType.cs, ISessionLogWriter.cs, LocalJsonLogWriter.cs (see §3.10) ProceduralGate/ ProceduralGateValidator.cs (thin wrapper: CanTransition/GetBlockReason delegate to EvidenceStateManager.CanAdvanceTo/GetItemBlockReason - the single source of truth for "is X allowed" stays on EvidenceStateManager) Testing/ GreyboxFlowTest.cs (see §3.8), NotificationPriorityTest.cs
```

Data, Scenes, Prefabs, Editor, Tools

Assets/_Project/Data/ Evidence/ EvidenceDefinition assets for CSI_Environment: EVD-014 (knife), EVD-015 (spatter), EVD-017 (phone), EVD-018 (bottle), EVD-019 (gloves), BODY-A (victim's body, examinedInPlace) EvidenceDefinitions/Tutorial/ a SEPARATE tutorial scenario's items (TUT-001..003, TUT-FP-001) - do not confuse with Data/Evidence/, different scenario DialoguePools/ STCS line-pool assets, one family per evidence item per moment Scenarios/ HypothesisCheckpoints_CSI_Environment.asset, GroundTruth_CSI_Environment.json (generated, see §3.10) Assets/_Project/Scenes/ Scenarios/CSI_Environment.unity the main, actively-developed scenario (this session's focus) Scenarios/Tutorial_ToolTest.unity tool-testbed scene (EvidenceNotifier.announceDiscovery=true here on purpose - see that field's own comment) Scenarios/HeacyASF.unity (not investigated this session - verify before assuming) Scenario_Basic01_Indoor.unity, Main.unity other scenes, not touched this session Assets/_Project/Prefabs/ Evidence/GrabbableEvidenceBase.prefab shared base for every grabbable evidence prop (see §3.9) Interaction/FlashlightTool.prefab, EvidenceTentTool.prefab, PhotographTool.prefab UI/ConnectionLine.prefab Assets/_Project/Art/Materials/ Mat_ChalkGuide_Uncovered.mat, Mat_ChalkGuide_Covered.mat, Mat_LetterTent.mat (deleted - see session history), others Assets/Editor/ ScenarioGroundTruthExporter.cs menu: Tools/VICSI/Export Scenario Ground Truth (see §3.10) McpHouseBuilder.cs, OpenableFurnitureSetup.cs, SceneLightingRealismSetup.cs scene-building editor utilities, not part of the runtime evidence system Tools/EvidenceScorer/ standalone .NET 8 console app, single file: Program.cs (see §3.10)

3. Core Architecture

3.1 EvidenceStateManager — the state machine

`Assets/_Project/Scripts/CaseFile/EvidenceStateManager.cs`. Singleton (`Instance`), holds a `Dictionary<string, EvidenceRecord>` keyed by evidenceId, populated in `Awake()` from the Inspector-assigned `sceneEvidenceDefinitions` list.

RequiredSequence — the one shared pipeline

```
NotFound -> Found -> PhotographedRaw -> ChalkMarked -> Marked -> PhotographedFinal
   -> Logged -> ReadyForCollection -> Collected -> Sealed -> Processed
```

This is a single, explicitly-ordered `EvidenceStatus[]` array, written out independently of the enum's own declaration order on purpose — a future enum insertion must not silently change gating. `IsValidNextStep` / `GetNextRequiredStatus` read this array, never the enum's ordinal directly. **Every item in the roster walks this exact same sequence** — per-item variation happens through flags and sub-step gates layered on top, never through a different sequence.

Shared-sequence steps vs. per-item flags — the key distinction

A step is in `RequiredSequence` only if *every* item must pass through it. Anything that only applies to *some* items is a flag on `EvidenceRecord` instead:

Flag	Meaning	Set by
`swabbingDone`	required blood trace has been swabbed	`MarkSwabbed`
`fingerprintingDone`	fingerprint processing complete	`MarkFingerprinted`
`isFlipped`	item physically turned over	`MarkFlipped`

These never appear in `RequiredSequence`; they gate specific transitions via the `IsBlockedByX` functions below.

TransitionResult — Applied / Duplicate / Violation

Every `MarkX` method returns this so a caller can tell "nothing to report" (`Duplicate` — a harmless repeat/backward call, e.g. a proximity trigger re-firing) from "blocked, tell the player why" (`Violation`). `Duplicate` is never logged as an error; `Violation` always is, and fires `OnEvidenceTransitionBlocked`.

The IsBlockedByX gate functions — what each gates NOW

> **Important for any future change:** a whole task this session ("Move Logged to Just Before Collection") moved these from gating `Collected` to gating `Logged`. If you ever see documentation or code elsewhere referencing these as gating `Collected`, it is stale — trust the current source below.

Function	Gates	Condition
`IsBlockedBySwabbing`	`PhotographedFinal -> Logged`	`requiresSwabbing && !swabbingDone`
`IsBlockedByOnSceneFingerprinting`	`PhotographedFinal -> Logged`	`fingerprintBeforeCollection && !fingerprintingDone`
`IsBlockedByFingerprinting`	`Sealed -> Processed`	`requiresFingerprinting && !fingerprintBeforeCollection && !fingerprintingDone` — the DEFAULT (post-Sealed) shape, unaffected by the reorder

Why Logged, not Collected: report-line selection (`RecorderTool`, gated on `CanTransition(id, Logged)`) must reflect everything actually discovered on that item — including a swab result or an on-scene fingerprint match — and logging feeds hypothesis-checkpoint availability downstream. All on-scene discovery has to land before `Logged`, not merely before the item is physically bagged.

`MarkFlipped` has its own additional rule, not expressed as an `IsBlockedByX` function: when an item `requiresFlipBeforeFingerprinting` AND `requiresSwabbing` (currently only the gloves), flip is further locked until `swabbingDone` — real procedure swabs the outside trace before ever turning the item over. This reuses `IsBlockedBySwabbing` 's own message for free (both gate `Logged`).

CheckReadyForCollection / CheckProcessed — the two auto-advances

`MarkLogged` calls `CheckReadyForCollection` , which immediately advances `Logged -> ReadyForCollection` (skipped entirely for `examinedInPlace` items, which terminate at `Logged`). `MarkSealed` and `MarkFingerprinted` both call `CheckProcessed` , which advances `Sealed -> Processed` the moment every precondition is already true — there is no distinct player action for "Processed," it just *is* processed once sealed and (where required) fingerprinted.

The decoupled FingerprintLabStatus side-track

`NotSent -> SentToLab -> Matched` , entirely separate from `record.status` / `RequiredSequence` . A print's chain-of-custody (Collected->Sealed->Processed) must never wait on a lab turnaround, so "sent"/"matched" live on their own 2-stage track (`MarkSentToLab` / `MarkMatched`), auto-triggered from `MarkFingerprinted` . The match result itself is a hardcoded placeholder (`PlaceholderMatchTargets` , currently EVD-014 → "Victim's brother", EVD-019 → "Victim's wife") standing in for a real suspect-matching system that doesn't exist yet.

examinedInPlace items

An item examined and logged where it lies, never bagged (currently only BODY-A, the victim's body). Its real chain is `Found -> PhotographedRaw -> ChalkMarked -> Marked -> PhotographedFinal -> Logged` , full stop — no `ReadyForCollection` / `Collected` / `Sealed` / `Processed` . It is marked with a LETTER tent (via the separate `LetterTentTool` , §3.3) instead of a numbered one, and is deliberately excluded from `CountAtOrAbove` (every caller of that aggregate assumes it reflects the numbered roster).

3.2 EvidenceProp / EvidenceDefinition — the data model

`EvidenceDefinition` (ScriptableObject, one asset per item under `Data/Evidence`) holds every piece of per-item authored data. Key fields, verified current on the live assets:

Field	Purpose
`evidenceId` , `displayName` , `relevance`	identity + scoring-only classification (never gates play)
`interactionRadius`	both the auto-Found proximity radius AND the tenting-match radius — deliberately one number
`chalkCircleRadius` , `chalkPointCount` , `chalkCompletionThreshold`	per-item chalk guide geometry (see §3.6); MUST be authored per item, no sensible universal default
`requiresSwabbing` , `collectedBySwab`	mutually exclusive: a precondition-on-a-different-item's-bagging vs. this item's OWN collection method (EVD-015 only)
`requiresFingerprinting` , `fingerprintBeforeCollection` , `requiresFlipBeforeFingerprinting`	see §3.1/3.5 — on-scene vs. default (post-Sealed) fingerprint shape
`examinedInPlace`	see §3.1
`reportLineOptions` , `minimumLinesRequired`	the drag-and-drop report content (§3.14); each `ReportLineOption` can carry `SelectionPrerequisite` s

`EvidenceProp` (MonoBehaviour, on the actual scene object) owns:

- A runtime-created trigger collider (sized from `interactionRadius`) that fires `MarkFound` automatically — suppressed entirely when `foundRequiresExternalTrigger` is true (used by marker sub-objects that reach Found some other way, e.g. via a UV reveal).
- `SurfaceNormalAxis` (LocalUp/LocalForward/LocalDown/LocalBack) — which local axis is "outward" for the chalk guide's own orientation. NOT uniform across the roster: flat-decal props (knife, spatter) need `LocalBack` ; the phone needs `LocalForward` ; upright props use the `LocalUp` default. Verify per-prop, never assume.
- `overrideChalkGuideTransform` + `chalkGuideLocalPosition/EulerAngles/Scale` — an opt-in per-prop escape hatch added this session so a prop whose own authored rotation/scale isn't a sane basis for a guide (BODY-A's body model sits at an odd 275° rotation) can still get a correctly-oriented guide. Off by default for every other item.

- The static registry (`EvidenceProp.All` , `FindById` , `FindNearestWithinRadius`) that every targeting system (tenting, photography) reads.

3.3 PlayerTool / the tool wheel

`Assets/_Project/Scripts/RoleSystem/PlayerTool.cs` . Abstract base for every equippable capability. Core contract:

- **Virtual equip, not physical pickup.** A tool starts hidden and inert; it only appears, attached to a hand, once selected from the tool wheel (`EquipToHand` / `Holster` , driven by `PlayerToolRegistry.ToggleEquip`). There is nothing sitting out in the world to walk up to.
- `equipLocalPosition` / `equipLocalEulerAngles` — per-tool offset applied relative to the hand anchor on equip. Edit directly in the Inspector under "Virtual Equip (tool wheel)."
- **The renderer/collider gotcha:** `SetEquippedVisualState` disables every collider under the tool's GameObject **in both equip states** (equipped and holstered) — equipping is exclusively the wheel's job; if colliders stayed enabled while equipped, a hand passing near the tool could grab it mid-air and fight the wheel's own transform pinning. Any tool with its own functional tip collider (brush, swab tip, tape tip) must explicitly re-enable/disable that specific collider in its own `EquipToHand` / `Holster` override.
- `PreferredHand` (defaults to Right) lets a tool request the left hand instead of the wheel special-casing it by `ToolType` .

`ToolType` is an explicit-int enum (same discipline as `EvidenceStatus`) — append, never renumber. Current roster: `Photographer, IOC, Sketcher, EvidenceCollector, Recorder, EvidenceMarker, Swab, FingerprintKit, ChalkMark, EvidenceFlip, FingerprintTape, LetterTentMarker` .

`PlayerToolRegistry` is the static directory (`GetTool` , `ToggleEquip` , `HolsterCurrent`) — adding a new tool needs zero changes here, it self-registers via `OnEnable` . `ToolWheelController` auto-enumerates every `ToolType` value; a role with no `PlayerTool` built yet just shows a dimmed, non-functional wedge.

> Selecting a *different* tool from the wheel always fully holsters whatever was equipped before — this is how switching from a two-handed tool (brush+jar) to a different one-handed tool (tape) empties *both* hands with zero special-casing anywhere.

3.4 The marker precedent pattern

Used by `EvidenceSwabRequirement` and `EvidenceFingerprintMarker` : a small physical sub-object with its *own* `evidenceId` field pointing to its PARENT's real roster id, found via its own static registry (`FindByEvidenceId`), gating the parent's own transition — explicitly **not** an independent `EvidenceProp` /roster item.

> **Why not just make it a second EvidenceProp?** This was tried first (an earlier design used an independent, co-located EvidenceProp for the knife's blood trace) and was **genuinely broken**, not merely awkward: `PhotographTool` 's targeting resolves by camera distance among every registered `EvidenceProp` — a stain a few centimetres from the knife's own pivot could not be reliably disambiguated from it, and photographing "the knife" kept crediting the trace instead. Making it a precondition marker sidesteps the conflict entirely: only one real EvidenceProp exists in that physical space.

Both marker classes: register only if `evidenceId` is non-empty; are found via `GetComponentInParent<T>()` from whatever collider a tool's touch/aim actually hit; carry no `EvidenceRecord` of their own.

3.5 On-scene fingerprint sub-step (FingerprintKitTool + FingerprintTapeTool)

Generalized this session off `EvidenceFingerprintMarker` — it used to be hardcoded to the knife alone (`KnifeEvidenceId` / `PrintMarkerId` consts, a single `isDusted` bool). Now:

- `IsDusted` lives **per-marker** (on `EvidenceFingerprintMarker` itself), not on the tool — a shared bool would have let dusting one print silently count as dusting a different item's.
- **Collision-based, not aim-and-click.** Loading dust needs an explicit trigger press (touch jar + press). Applying dust and lifting need *no* press at all — the dusted brush touching a print's own trigger "TouchZone" collider fires `TryDust(marker)` the instant contact happens, the same way a real brush swiped across a surface deposits powder immediately. The print's TouchZone is a *separate* trigger collider from its other, solid, non-trigger one — that second one still has to stay solid for `FlashlightTool` 's own occlusion-style raycast.

- `FingerprintTapeTool` is a separately-equippable tool (own `ToolType.FingerprintTape`) whose trigger handler is a thin forwarder into `FingerprintKitTool.TryLift(marker)` — `TryLift`'s implementation deliberately stays on `FingerprintKitTool` (single source of truth), not moved to the tape tool.
- Currently used by both EVD-014 (knife) and EVD-019 (gloves) — both have their own `EvidenceFingerprintMarker` child object with a UV-hidden print, revealed via `UVRevealable.TriggerReveal`. The gloves' fingerprint marker additionally requires `isFlipped` before dusting will succeed (checked inside `TryDust`).
- `FingerprintStationTool` is now used *only* for the DEFAULT (post-Sealed) shape — currently EVD-017/018. It has real failability (dust-coverage window, over/under-dusting smudges and resets).

3.6 Scene-authored vs runtime-generated objects

This is the reusable pattern for the still-pending conversions listed in §6. Worked example: `EvidenceChalkGuide`.

The problem

The chalk guide used to be built entirely from scratch in code (`new GameObject`, `AddComponent<LineRenderer>`, generated materials) every time an item reached `PhotographedRaw`. That makes it impossible to hand-tune in the Editor — any edit is invisible until Play mode, and gets silently rebuilt from formula every session anyway.

The pattern (two co-existing paths)

1. **Template + clone, for parametric per-item objects.** One authored template (`ChalkMarkTool.chalkGuideTemplate`, a real inactive scene GameObject with the max point/segment count pre-built and real material assets assigned) is `Instantiate`d and *configured* per item — not built from scratch. The template itself carries the `EvidenceChalkGuide` component (not `AddComponent`'d after cloning) specifically so Inspector-assigned fields (the two materials) survive the clone — a component added via `AddComponent<T>()` at runtime always starts with default field values, it does not inherit anything.
2. **Direct authoring adoption, for a specific item.** `SpawnFor` first checks for an already-existing `"ChalkGuide_" + evidenceId` child directly under the prop — if a human (or an AI) hand-placed one, it is *adopted* (`InitializeFromAuthored`) instead of cloning the template. Currently used for EVD-017 (phone) and BODY-A.

> **A real bug found and fixed this session:** the first version of the adoption path still called the regular `Initialize`, which unconditionally recomputes every point's position from the `chalkCircleRadius` formula — silently discarding any hand-edit the moment the guide (re)spawned. `InitializeFromAuthored` is a genuinely separate method that touches only runtime-driven state (current material, current toolEquipped-gated visibility) and never repositions a point, never resizes a segment, and reads the point/segment *count* from however many actually exist rather than trusting the data.

ChalkGuideSegmentLink — why plain Transform-parenting doesn't work

A `LineRenderer`'s endpoints are **static baked coordinates** set via `SetPosition`, completely independent of Transform hierarchy. Parenting a point under its segment would move the point when you drag the segment, but the *drawn line* would not follow — it stays at its old baked position. The fix: a small `[ExecuteAlways]` component on each segment holding live Transform references to its start/end points, redrawing every `LateUpdate` (in both Play mode and the Editor). This is what makes hand-dragging a point in the Scene view visibly update the connected lines in real time, with no Play mode needed.

A second real gotcha hit while authoring the phone/body guides

`GameObject.Find` cannot find inactive objects — when the freshly-created template was momentarily inactive, a follow-up `GameObject.Find("ChalkGuideTemplate")` call failed. Use `FindObjectOfType<T>(includeInactive: true)`, or hold the reference from the same call that created/deactivated it.

> **For the pending conversions** (checkpoint UI fallback panel, completion screen, letter-tent stub, flip-tool stub, fingerprint station pad) — first determine whether the object is a true singleton (most of these are) or genuinely parametric per-instance like the chalk guide. A singleton only needs the "direct authoring adoption" half of this pattern (author it once in the scene, have the script find-and-toggle it instead of building it) — no template/clone machinery needed. Also check whether the object is ever parented under a `PlayerTool` (the flip-tool stub and fingerprint station pad plausibly are, or at least visually represent held/interactive props) — if so, `SetActive` is the correct hide mechanism, NOT renderer-disabling, because `PlayerTool.SetEquippedVisualState` re-enables/disables renderers on its own schedule tied to equip state and will fight a hand-managed renderer toggle.

3.7 PhotographTool aim-target resolution

Every frame while aiming, `UpdateAimTarget` scans `EvidenceProp.All` and resolves a single `CurrentAimTarget` under three filters, in order:

1. **Must be at-or-past Found.** An undiscovered item (e.g. a not-yet-revealed print marker) can never win, regardless of distance/framing — without this, an undiscovered co-located item could steal the aim target from the one the player actually meant to shoot (confirmed live as the original cause of "can't photograph the knife").
2. **In range and in frustum.** Within `maxDistance`, and its pivot's viewport point must land in [0,1] on x/y with positive z.
3. **Unoccluded.** One raycast per candidate toward its own pivot; the first thing hit must be that same item, or it's disqualified (no partial-occlusion tolerance).

Tie-break among survivors: as of this session, the candidate closest to **viewport center** wins — not the one physically nearest the camera. This was changed after a confirmed bug: two items sitting close together (e.g. the blood spatter 0.34m from the body) would have the nearer one hijack the shot even when the player was framed dead-center on the other. `TakePhoto` only ever acts on the single resolved `CurrentAimTarget` — a shutter press has never been able to advance more than one item at once; the bug was about *which one* got picked, not multiple-at-once.

3.8 GreyboxFlowTest

`Assets/_Project/Scripts/Testing/GreyboxFlowTest.cs`. A checked-in, re-runnable regression test — exists specifically because an earlier ad-hoc walkthrough (typed into the Editor console once and never saved) silently stopped being valid when the sequence changed, with nothing to fail. Run via:

```
FindFirstObjectByType<GreyboxFlowTest>().RunFullFlow()
```

(from Play mode, via MCP `execute_code`, or tick `runOnStart`). Drives every roster item (currently `EVD-014, EVD-015, EVD-017, EVD-018, EVD-019`) through the full lifecycle, asserting at each step; a line starting with `FAIL` is a real regression. Where a real tool exists, it calls the tool's own public `TryX` method (e.g. `SwabTool.TrySampleRequiredSwab`, `FingerprintKitTool.HandleBrushTriggerEnter`) rather than the bare `EvidenceStateManager.MarkX`, so it also proves the real call path, not just the underlying gate.

> **What it explicitly does NOT cover** (confirmed this session, a real blind spot): it drives the state machine directly and does not simulate VR input at all — it never exercises a tool's actual `Update()` loop or real button-press detection (`InputAction.WasPressedThisFrame()`). A bug purely in real trigger/input wiring can pass this test cleanly while still being broken on a real headset. When debugging a "nothing happens when I press the trigger" report, this test alone cannot rule that class of bug out — you have to simulate the real equipped-tool + real hand-anchor position and call the actual private input-driven method via reflection, or get on a real device.

State does not reset between runs within the same Play session — always stop and restart Play mode before re-running for a clean result. Each run also regenerates BODY-A driving manually where needed (it is intentionally excluded from the main roster loop, since `examinedInPlace` items are excluded from `CountAtOrAbove`).

3.9 GrabbableEvidenceBase.prefab

`Assets/_Project/Prefabs/Evidence/GrabbableEvidenceBase.prefab`. The shared base every physically-collectible evidence item (knife, phone, bottle, gloves) is a real connected instance of — verify with `PrefabUtility.IsPartOfAnyPrefab` / `GetPrefabAssetPathOfNearestInstanceRoot` before assuming an item is a loose GameObject.

Base prefab provides: `MeshFilter, MeshRenderer, BoxCollider, Rigidbody (non-kinematic, gravity on by default), Grabbable, HandGrabInteractable, GrabInteractable, EvidenceProp, ToggleGrab`.

Every real instance adds (as per-instance component overrides, not part of the prefab asset itself — confirmed via `PrefabUtility.GetAddedComponents`): `EvidenceGrabGate` + `RightHandOnlyFilter` (wired into both interactables' `_interactorFilters`). Every instance also overrides the `Rigidbody` to `isKinematic=true, useGravity=false` (the prefab's own default is the opposite).

Two visual conventions coexist among instances — match whichever fits the item's own mesh setup, don't guess:

Knife/phone style	Bottle/gloves style

Root's own `MeshFilter` / `MeshRenderer` / `ToggleGrab` REMOVED; a separate `Visual` child carries the real mesh	Root KEEPS the prefab's default `MeshFilter` / `MeshRenderer` / `ToggleGrab`, mesh assigned directly on the root, no Visual child

> A stray, non-prefab-connected copy of these components manually bolted onto an item (rather than a real `PrefabUtility.InstantiatePrefab` instance) is a real mistake to avoid — it works functionally but loses prefab-level maintainability and was explicitly corrected for the gloves this session after being built that way first.

3.10 Assessment pipeline

SessionLogger (`Assets/_Project/Scripts/Logging/`) writes every meaningful player action as a `SessionEvent` (typed via `SessionEventType`) to a local JSON file via `LocalJsonLogWriter`, at `Application.persistentDataPath` (Windows: `AppData/LocalLow/DefaultCompany/My project (1)/session_<guid>_<timestamp>.jsonl`). This is the append-only ground truth of what actually happened — state rewinds (e.g. tent reclaim) never delete or rewrite prior entries.

ScenarioGroundTruthExporter (`Assets/Editor/ScenarioGroundTruthExporter.cs`, menu `Tools/VICSI/Export Scenario Ground Truth`) reads `EvidenceStateManager.sceneEvidenceDefinitions` directly via `SerializedObject` — no second hand-maintained roster list — and writes `Data/Scenarios/GroundTruth_CSI_Environment.json`: the roster (id/name/relevance), hypothesis checkpoint correct-answer keys, and per-item report-line accuracy keys. Fully generic over roster size/content; re-run this after adding or removing any evidence item.

EvidenceScorer (`Tools/EvidenceScorer`, standalone .NET 8 console app, single file `Program.cs`) takes a session log + the ground truth JSON and computes: critical/relevant recall, precision, distractor fall-rate, reasoning accuracy, documentation accuracy. Run from that directory:

```
dotnet run --no-build -- <sessionLogPath> <groundTruthPath> [outputPath]
```

3.11 InteractionFeedback

`Assets/_Project/Scripts/Feedback/InteractionFeedback.cs`. Exactly two static channels, deliberately opposite in what they signal:

- **`Confirm(Transform pulseTarget = null)`** — "the game registered what you did," fires identically whether the action was correct or not (marking a decoy confirms exactly like marking the real weapon — anything else would leak the answer). Haptic pulse + generated tone + optional scale-pop on `pulseTarget`.
- **`Blocked(Transform pulseTarget = null)`** — "the game refused to do that," reserved strictly for procedural refusals (skipping a step) that the player MUST be able to distinguish from an accepted action. Never used to signal a wrong identification.

Audio is generated procedurally at runtime (`BuildTone`) rather than shipped as clip assets, specifically so a missing-asset reference can never silently produce nothing. A confirmed real gap this session: `EvidenceFlipTool.TryFlip`'s success case called bare `InteractionFeedback.Confirm()` with no toast at all — unlike every other successful action in the project. Fixed by adding an explicit `NotificationManager.Notify` call; keep this consistency in mind for any new tool's success path.

3.12 Hypothesis / Deduction system

`HypothesisCheckpointManager` watches `EvidenceStateManager.OnEvidenceStatusChanged` / `OnReportLineSelected` / `OnFingerprintLabStatusChanged` and fires authored checkpoints (data: `HypothesisCheckpointSet`) once their trigger condition is met (a count threshold, or a specific submission-initiated trigger for the final one). Each checkpoint offers reasoning options, some gated by `SelectionPrerequisite` (§3.14). Every answer is logged with the evidence state at that moment — the point of this system is the data captured, not the screen shown. `HypothesisCheckpointUI` has a runtime-generated fallback panel used only when no `panelRoot` is authored (a pending scene-authoring conversion, §6).

Separately, `EvidenceBoardController` / `EvidenceNode` / `ConnectionLine` / `DeductionScorer` implement a corkboard-style evidence-connection board — a distinct system from the hypothesis checkpoints; do not conflate the two when extending either.

3.13 STCS system

`STCSManager` fires authored dialogue moments (simulated teammates commenting) off the same `EvidenceStateManager` events, keyed by trigger id (e.g. `"evidence_019_flip_hint"`, built generically from the evidenceId's digits via `PlayerTool.FireEvidenceSTCSTrigger`). A real, fixed bug this session: the gloves' flip-hint trigger used to fire at `ReadyForCollection`, which — after the Logged reorder (§3.1) — was already *after* the point flipping had to happen, making the hint useless. Now fires at `PhotographedFinal`, matching when flip actually becomes available. **Any future reorder of when a sub-step becomes available must be checked against every STCS trigger tied to that same moment.**

3.14 RecorderReportUI & cross-item unlocks

`RecorderTool` (aim-and-activate) opens `RecorderReportUI`'s drag-and-drop panel for an item once it's `PhotographedFinal`; confirming is what actually calls `MarkLogged` (gated a second time at confirm, in case state changed while the panel was open). Each `ReportLineOption` can carry a list of `SelectionPrerequisite`s (§3.4-adjacent, see `SelectionPrerequisite.cs`) — e.g. the knife's "may be the murder weapon" line requires BODY-A's wound-description line to have already been selected. `SelectionPrerequisite.AllSatisfied` is the single shared implementation, called identically from `RecorderReportUI`, `HypothesisCheckpointManager`, and `HypothesisCheckpointUI` — do not duplicate this check anywhere else.

4. Established Conventions and Gotchas

Scene `[SerializeField]` values silently override code defaults. Changing a field's default value in C# does nothing for an object already serialized into a scene with the old value baked in — the scene's own data wins. Always verify live via `SerializedObject` after a change like this, don't trust the code alone.

`EditorUtility.SetDirty` and the MCP `manage_scene action:"save"` tool do NOT reliably persist scene changes. The robust pattern, used consistently throughout this session: call `UnityEditor.SceneManagement.EditorSceneManager.MarkSceneDirty(scene)` then `EditorSceneManager.SaveScene(scene)` directly via `execute_code`, **always** followed by a verification reload (`manage_scene action:"load"`) before trusting the edit persisted.

Play Mode changes never persist. Any edit made to a scene GameObject while `Application.isPlaying == true` is discarded the instant Play mode exits, full stop — there is no exception. Scene edits (including material/component wiring) must be made in Edit mode.

`GameObject.Find` cannot find inactive objects. Use `FindObjectOfType<T>(includeInactive: true)`, or keep the reference from whichever call created/deactivated the object.

CodeDom (the default MCP `execute_code` compiler) does not support local functions or a `using` statement mid-method-body. Write helper logic as a lambda (`System.Action<...>` / `Func<...>`) instead of a local function, and use fully-qualified type names (`Oculus.Interaction.Grabbable`, `UnityEditor.SceneManagement.EditorSceneManager`) instead of a `using` directive, which is invalid inside a wrapped method body.

URP vs. the Built-in "Standard" shader. This project is URP; the legacy Built-in "Standard" shader renders as the magenta/pink error shader under URP. Any generated placeholder material must use `Shader.Find("Universal Render Pipeline/Lit")`. Confirmed root cause of at least two real bugs this session (a letter-tent stub, and the gloves' own placeholder material).

The notification toast queue is asynchronous. `NotificationManager.Notify` enqueues; the visible label only advances as the display coroutine's own timer allows. Reading the label text in the same synchronous call as the `Notify` (or the same MCP round-trip) will show whatever was ALREADY displaying, not the new message — you must let real time pass (a separate tool call) before checking.

Working discipline

- **"Preserve all existing gating/behavior" is enforced by live re-verification, not code review alone.** Every reorder or refactor this session (the Logged move, the fingerprint generalization, the PhotographTool tie-break fix) was followed by entering Play mode, driving the exact repro scenario, and confirming the before/after numerically — not just reading the diff.
- **Ask before an irreversible architecture fork.** When a request has two materially different correct implementations (e.g. "should the chalk guide be one template-per-item or six hand-authored objects," "should moving a point re-parent it or live-follow it") this session consistently asked a clarifying question (via a structured choice) before committing, rather than guessing and rebuilding later.
- **Run `GreyboxFlowTest.RunFullFlow()` after any change touching the state machine, gating, or a tool's public `TryX` method**, in a freshly-restarted Play session (state does not reset between runs otherwise).

5. How to Extend This Project

Add a new evidence item

1. Create an `EvidenceDefinition` asset under `Data/Evidence/` (menu: *VR-CSI/Evidence Definition*), fill in `evidenceId`, `displayName`, `relevance`, `interactionRadius`, `chalkCircleRadius` / `chalkPointCount`, and whichever of `requiresSwabbing` / `requiresFingerprinting` / `fingerprintBeforeCollection` / `requiresFlipBeforeFingerprinting` / `examinedInPlace` apply.
2. Add the physical scene object: either a real `GrabbableEvidenceBase.prefab` instance (§3.9) if it's meant to be collected, or a plain object with just `EvidenceProp` if it isn't (e.g. `examinedInPlace`). Set `evidenceId` and `SurfaceNormalAxis` to match its actual resting rotation.
3. Add its own sub-marker children if needed (a real `EvidenceSwabRequirement` or `EvidenceFingerprintMarker` child, matching §3.4/3.5) — never a second independent `EvidenceProp` co-located with the parent.
4. Register the new `EvidenceDefinition` in `EvidenceStateManager.sceneEvidenceDefinitions` (Inspector list on the scene's `EvidenceStateManager` component) — verify via `SerializedObject`, save+reload.
5. Re-run `Tools/VICSI/Export Scenario Ground Truth` and confirm the new item appears with the right relevance.
6. Add it to `GreyboxFlowTest.evidenceIds` (a `[SerializeField]` array on the scene's `GreyboxFlowTest` component — edit via `SerializedObject`, not just the C# default array, per the scene-override gotcha) and drive it through `RunFullFlow()`.

Add a new tool

1. Add a new value to `ToolType` (append, explicit int, never renumber existing values).
2. Create a new `PlayerTool` subclass following the aim-and-activate or collision-based conventions already in `Assets/_Project/Scripts/Interaction/` — look at `EvidenceFlipTool` (simplest aim-and-activate) or `FingerprintTapeTool` (simplest collision-based) as templates.
3. Wire it into the scene as a real GameObject with the tool component + whatever physical prop/collider it needs; set `equipLocalPosition` / `equipLocalEulerAngles`.
4. Optionally add a display name/description in `ToolWheelController.ToDisplayName` / `DefaultDescription` — falls back to the raw enum name/blank otherwise, which still works, just reads plainly.
5. Remember the collider-disable-in-both-states gotcha (§3.3) for any functional tip/contact collider.

Add a new on-scene sub-step marker

Follow the marker precedent (§3.4) exactly: a small MonoBehaviour with its own `evidenceId` field pointing to the parent, a static registry with `FindByEvidenceId`, never a second `EvidenceProp`. Gate the relevant transition through a new (or extended existing) `IsBlockedByX` function on `EvidenceStateManager`, and check the message it produces flows correctly through `ProceduralGateValidator.GetBlockReason` to whichever tool's refusal path calls it.

Standard verification loop for any change

1. Compile-check: `refresh_unity` then `read_console action:"get" types:["error"]`.
2. Enter Play mode, drive the exact scenario via direct `EvidenceStateManager` /tool method calls (matching how `GreyboxFlowTest` itself drives things — see §3.8's caveat about what this does NOT cover).
3. For anything visual/positional: also verify via a real equipped-tool + real hand-anchor-position simulation, not just a manually-teleported aim origin, before concluding "it works."
4. Stop Play mode, re-enter fresh, run `GreyboxFlowTest.RunFullFlow()`, confirm `PASS`.
5. For any scene edit: save via `MarkSceneDirty` + `SaveScene`, then reload the scene and re-verify the specific field/value persisted before reporting success.

6. Current Known Gaps / Open Items

Pending scene-authoring conversions

Following the pattern in §3.6, still runtime-generated as of this document:

- `HypothesisCheckpointUI` 's fallback panel (only built when no `panelRoot` is authored — likely just needs direct authoring, singleton case, no template/clone needed).
- `CompletionScreenUI` 's equivalent fallback (same shape).
- The letter-tent stub's remaining runtime pieces, if any beyond what was already converted this session — verify current state before assuming untouched.
- The flip-tool stub — also flagged this session as having an unusually cluttered component set (a stray `EvidenceProp` / `Grabbable` / `ToggleGrab` / `Light` apparently inherited from being built off the wrong source object). Worth auditing/cleaning regardless of the scene-authoring conversion.
- The fingerprint station pad's generated coverage-indicator visual.

For each: first determine singleton vs. parametric-per-item (§3.6), then check whether it sits under a `PlayerTool` hierarchy (changes the correct hide mechanism to `SetActive`).

Gloves do not visually rotate on flip

Deliberate by original design (`MarkFlipped` is documented as "genuinely gated, not cosmetic," a pure logic flag) but flagged as a real UX gap this session after a "flip isn't working" report turned out to really mean "flip has zero visible feedback." A toast notification was added as a minimal fix; a real visual rotation of the gloves mesh was explicitly NOT built (would need to decide which axis/direction, and whether the fingerprint print marker's own position needs to move to the new "up" side). Still open if wanted.

Unresolved "flip aint working" report

The user reported the flip tool doing nothing at all on a real headset (no toast, no haptic, no sound) even with every gating condition met. Exhaustively re-verified this session via every testable path — direct method call, full raycast against the rebuilt gloves object, and (most rigorously) genuinely equipping the tool via the wheel and moving the REAL `OVRCameraRig.rightHandAnchor` (not a manually-teleported aim origin) before calling the actual private `TryFlipAimedTarget()` method — and it succeeded every time. This rules out the gating logic, the raycast/targeting logic, and the equip-follow behavior as the cause. Since `WasPressedThisFrame()` reads real Input System device state that cannot be faked from script, this could not be fully ruled out as an input-binding or real-environment-occlusion issue specific to the actual headset session. **Next step if revisited:** ask whether a different aim-and-activate tool (e.g. `RecorderTool` , identical input binding) works from the exact same real position — that isolates whether the problem is flip-specific or environmental.

Other loose threads

- `EvidenceTentPickup` / `EvidenceTentPickupCoordinator` (tent reclaim) is intentionally disabled (`TentPickupEnabled = false` constant in `EvidenceTentTool`) — placed tents currently cannot be picked back up at all in the live build, regardless of the marked item's status. Known consequence: at most `tentVisuals.Length` (6) numbered tents can ever be placed in one session, since slots never free up.
- The `Scenes/Scenarios/HeacyASF.unity` scene was not investigated this session — verify its purpose/state before assuming it's related to the CSI_Environment work.
- The `Data/EvidenceDefinitions/Tutorial/` assets (TUT-001..003, TUT-FP-001) belong to a separate tutorial scenario, not the CSI_Environment roster — do not conflate the two when searching for "all evidence items."
- A handful of unrelated, pre-existing console warnings recur across every Play session in this project (repeated "BoxCollider does not support negative scale or size," various XR/OpenXR/audio-driver messages) — confirmed multiple times this session as pre-existing background noise, not caused by any of the changes described here. Don't chase these unless a specific new one appears.